Assignment 1 Report
DD2380

Group 24
Max Decman Gawtam Ramesh
decman@kth.se gawtam@kth.se
18.06.2001 12.10.2001


Abstract

The deployment of autonomous systems relies on path planning, which is essential for the safe and effective operation of multi-agent platforms in rapidly changing environments. With the increasing use and investment in autonomous vehicles for delivery and transportation, addressing the difficulties of traversing complex environments is critical. In this work, we propose a path planning framework for a car and a drone, implemented within the Unity simulation environment. Both are two-dimensionally constrained. Our method combines an adaptation of Hybrid A* for pathfinding with a waypoint tracking system for local navigation. Together, they allow both the car and the drone to follow viable paths that adhere to their kinematic constraints. We demonstrate fast and reliable navigation in a range of scenarios through a series of simulated tasks. We also critically evaluate our methodology against advanced approaches from other groups, highlighting our shortcomings and discussing potential solutions. Our results provide insights into autonomous navigation systems and further our understanding of the trade-offs in agent path planning.

1 Introduction

Autonomous vehicles, like self-driving cars and delivery drones, are becoming integral to modern logistics and transportation. In order to operate safely and effectively, such systems must be capable of continuously determining efficient routes from one location to another while avoiding obstacles. This process, known as path planning, involves more than simply choosing a route. It must account for the physical limitations of each vehicle, including steering constraints and turning radius.

Conventional path planning strategies frequently rely on classical algorithms such as Dijkstra's and A* which are designed to efficiently find routes on a two-dimensional grid. However, these methods are not designed for the continuous and dynamic behavior of vehicle motions. As a result, they can produce suboptimal paths that ignore these dynamics. For instance, a self-driving car might be provided with a seemingly optimal path on a map that requires unrealistic sharp turns that cannot be applied in practice. To address these limitations, more modern techniques like Hybrid A* [2] have been developed, integrating vehicle dynamics to generate smoother and more realistic trajectories.

Despite these improvements, challenges remain. Current methods can struggle in dynamic real-world settings, narrow pathways, and unexpected, possibly moving, obstacles. Addressing these limitations enables more reliable navigation in crowded urban areas and improves safety among autonomous agents. Our work aims to implement the Hybrid A* planning method for a car and a drone operating in a two-dimensional space with modifications for computation time and a waypoint navigation system.

1.1 Contribution

We demonstrate that our adaptation of Hybrid A* can effectively generate smooth trajectories for a kinematically constrained vehicle with minimal post-processing within the Unity environment. Additionally, we show how agents with different movement constraints can be coordinated in various environments. Our findings provide insights into the strengths and limitations of Hybrid A* in constrained navigation.

1.2 Outline

This report is structured to provide a comprehensive overview of our multi-agent path planning implementation, from the underlying motivation to a critical assessment of our results. We begin by discussing the fundamental

challenges of autonomous path planning by introducing related path planning work in section 2, placing the approach we use in the context of existing research. The proposed methodology in section 3 describes our Unity implementation, including how the car and drone use Hybrid A* and follow the waypoints which were generated. We then analyze the performance of our navigation strategy in section 4 using simulation results of different terrains to assess our approach. We compare these results with implementations of other groups and highlight shortcomings. Moreover, section 5 discusses the limitations and failures of our approach and suggestions for improvement. Finally, we conclude by summarizing our findings in section 6 and propose an outlook for future topics in this environment.

2 Related work

Developing reliable path planning methods is a central challenge in autonomous navigation. Over the years, researchers have proposed a wide range of techniques to compute collision-free trajectories that are both efficient and practical for real-world vehicles. These approaches can be categorized into two graph-based search methods and sampling-based planners. Graph-based strategies work on discrete maps to determine the best routes. Sampling-based techniques explore continuous state spaces, but they may provide pathways that need additional post-processing in order to be useful. The following subsections discuss the two types in more detail with their variations. The emphasis is on Hybrid A*, as it forms the basis of the proposed solution.

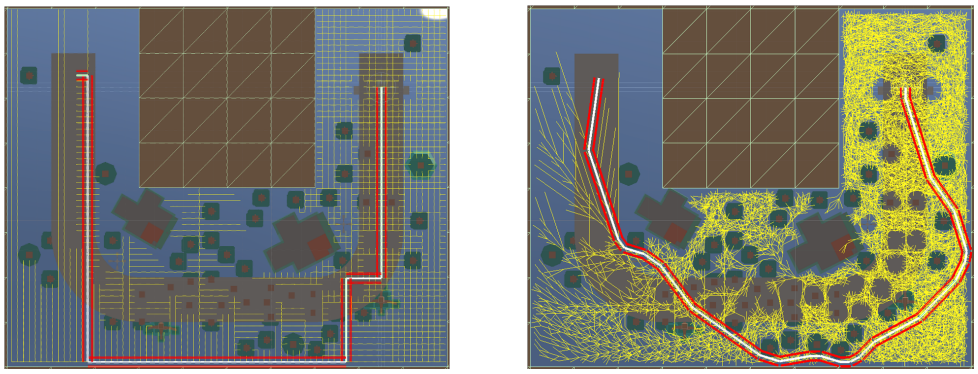
2.1 Graph-based Algorithms

Path planning in discrete terrains is commonly framed as a graph search problem, where the environment is modeled as a graph of nodes and edges. Every node represents a unique spatial position and edges describe the transitions between them. A path planning algorithm tries to find the best edge sequence to minimize the cost function between a start node and a goal node.

Classical A* is used on a discrete grid map where every node represents a unique spatial location. The cost function $f(n) = g(n)+h(n)$ is minimized to find the shortest path, where $g(n)$ is the total cost from the start node to node n. The cost to the goal is estimated using a chosen heuristic $h(n)$. At each iteration, A* selects the node with the lowest $f(n)$ from a set and expands it by generating its neighboring nodes. This node expansion involves evaluating up to four possible moves, excluding diagonals, at each node. Although this

strategy guarantees optimality under appropriate conditions, the grid-based representation often leads to paths with abrupt turns that are impractical for vehicles.

A solution for this is proposed by Daniel et al. [2] with the Hybrid A* approach. It addresses the limitations of the classical approach by extending the state space to incorporate continuous variables such as the vehicle's orientation. Rather than limiting movement to neighboring grid cells, Hybrid A* uses *motion primitives* for movement, which are precomputed trajectories derived from a kinematic model that respect constraints like minimum turning radius or maximum steering angle. Using motion primitives during node expansion results in feasible paths for the chosen model. The direct comparison can be seen in Figure 1. The white and red lines are the final path, with the yellow lines being the motion primitives during search. These realistic paths come at the cost of increased computational complexity because each node now considers both *position and orientation*, and the algorithm must evaluate several motion primitives at every step.



(a) Regular A* (b) Hybrid A*

Figure 1: Comparison of computed paths

Another variation of A* is Theta*, as introduced in [1]. This approach relaxes grid constraints by enabling any-angle path planning. During its node expansion, line-of-sight checks are used to determine if a direct connection exists between a node's parent and another successor. If the path is not blocked, the algorithm skips the intermediate nodes. The line-of-sight calculations increase computation time, particularly in denser environments.

The Jump Point Search [4] approach focuses on reducing the computation time of A* by *jumping* over redundant nodes, expanding to nodes where obstacles force a change in direction. It enhances efficiency in larger, open environments. The performance gains are less pronounced in more complex or dense obstacle settings.

2.2 Sampling-based methods

Sampling-based path planners probabilistically explore the configuration space by expanding a tree or graph. For this work, we review sampling-based methods in the RRT family, analyzing their technical foundations.

Introduced by LaValle [6], RRT is designed for efficient exploration of high-dimensional spaces. The algorithm incrementally constructs a tree by randomly sampling points in the configuration space and extending the nearest node toward the sample. At each iteration, a random configuration is generated and the nearest valid node in the tree is connected to the node it generated from with a straight line. While RRT is probabilistically guaranteed to find a path if one exists, it does not optimize path quality. Hence, the produced trajectories are jagged.

To address the suboptimality of the RRT paths, [5] incorporate a *rewiring* step resulting in RRT*. After adding a new node, the algorithm evaluates nearby nodes within a cost-defined radius and reconnects them to the newly added node if doing so reduces their cumulative cost. This cost function ensures asymptotic optimality, converging to the shortest path as the number of iterations increases. However, the rewiring step increases the computational overhead. RRT* generates smoother paths compared to regular RRT but still struggles in systems with movement constraints.

cxmmKinodynamic RRT* [9] integrates system kinematics directly into the planning process, similar to Hybrid A*. Instead of straight-line connections, the nodes are expanded using trajectories derived from the system's kinematic model. This method ensures feasibility for real-world systems but increases computational complexity greatly.

Another extension is Informed RRT* [3]. This approach improves convergence speed by focusing sampling within a specific heuristic region that could shorten the initial best found path. This reduces the exploration of irrelevant regions. While it has the same time complexity as RRT*, informed RRT* reduces computation time in cluttered environments by up to 90%.

3 Proposed method

Reviewing the pros and cons of various graph-based and sampling-based approaches, along with the vehicle constraints and the diverse terrains in this project, we chose to experiment with Hybrid A* and Kinodynamic RRT*. These methods were selected because they incorporate the motion model of the vehicles, a crucial factor for achieving feasible and realistic paths.

After testing both methods in Unity, we finalized our approach with Hy-

brid A* due to its computational efficiency and consistent performance. The deterministic nature of its solutions also made post-processing more reliable, reducing the need for extensive smoothing.

3.1 Path Planning

For our path planner, we propose a modified Hybrid A* inspired by the description in [7]. We discretize the terrain and traverse the state space using motion primitives. We precompute trajectories using a simplified kinematic model derived from the dynamic bicycle model [8] for our motion primitives. In our model, the vehicle states are represented by its position in the two-dimensional plane (x, z) and heading θ. A single motion primitive updates the state as follows:

$$x' = x + s\cos\theta, \qquad z' = z + s\sin\theta, \qquad \theta' = \theta + \frac{s}{L}\tan(\delta), \tag{1}$$

where s is the step size, set to approximately 3.5 units, δ is the steering angle and L represents an effective wheelbase length set to 2. The steering angle is selected from 11 evenly spaced values over a 50° range. The drone's steering is discretized into 9 values over a 60° range with a step size of 5. This model respects constraints such as the minimum turning radius without modeling lateral forces or slip angles. The drone uses the same kinematic structure to ensure smooth paths, allowing for faster speeds without exceeding the path in turns.

Each motion primitive extends the state forward in the search tree. A discrete obstacle map with a 1-unit grid resolution is used to check for collisions, while the search itself is discretized at 2-unit intervals to reduce computational complexity. During node expansion, the algorithm verifies path viability by ensuring that a simulated width, larger than the vehicle's actual width, is traversable. The width is set to 3 for the car and 5.4 for the drone. A hashing system stores expanded nodes by discretizing orientation into 18 and 36 bins, respectively, reducing potential redundant computations.

The heuristic function is based on the Euclidean distance to the goal with an added penalty for sharp turns, encouraging smoother, high-speed motion. Minimal post-processing is performed on the generated trajectory by removing intermediate waypoints that lie on a straight line, retaining only the endpoints for computing acceleration and deceleration profiles.

3.2 Waypoint Tracking

The next step is to make sure the agents precisely follow the path our Hybrid A* algorithm has calculated. We use a waypoint tracking system designed for the car and drone that is adapted to their unique dynamics.

For the car, the system continuously updates the vehicle's position relative to the next waypoint along the route. The process is as follows:

- First a vector is calculated from the car's current position to the current waypoint.
- This vector is then transformed into the car's local coordinate system to determine the required steering adjustment.
- The forward component of this vector sets the acceleration, while the lateral component defines the steering command.
- When the car comes within 8 units of the waypoint, the system automatically shifts focus to the next waypoint.
- If the car is not moving, a fallback mechanism reverses acceleration and steering for 2 seconds to reposition the vehicle.

This approach allows the car to follow the path closely, maintaining high speeds on long straight segments and decelerating appropriately for curves. In our implementation, the car's base speed is set to 10 units, with an upper limit of 22 units when the next waypoint is more than 30 units away.

The drone's tracking system builds on the car's method but is adjusted slightly. It follows different instructions after converting the direction vector into the drone's local coordinate frame:

- The drone accelerates directly toward the waypoint using the given directions in both axes
- When the distance to the next waypoint is less than 11 units, the system reverses acceleration to decelerate. The closer the drone gets, the stronger the deceleration.
- Once the drone is within 4 units of the waypoint, it shifts its focus to the next waypoint.

This adaptive braking mechanism ensures that the drone decelerates enough to avoid overshooting the path. This provides precise control even in obstacle-dense environments.

In summary, our waypoint tracking systems enable both the car and the drone to follow their planned trajectories smoothly and efficiently, adapting dynamically to changing conditions and waypoint distances along the route.

4 Experimental results

4.1 Experimental setup

Our algorithm was tested across a diverse set of terrains consisting of different challenges. We first ran tests on five predefined maps, followed by a second evaluation in a competition setting featuring five previously unseen maps. Performance was assessed based on completion time, with lower values representing better performance. In the competition phase, we were allowed to adjust parameters to optimize our completion times. If a system failed to complete a course, it was recorded as DNF (did not finish). In the following analysis, we examine the results from both test sets. We identify limitations and discuss alternative approaches adopted by other groups to address similar challenges. For the following comparisons, we chose a subset of representative groups that performed better or similarly to our approach.

4.2 Analysis of Outcome

Group	**Car**	**Drone**	**Total**
6	184.0	204.6	388.6
24	**196.9**	**225.4**	**422.3**
4	157.1	266.3	423.4
17	181.4	313.7	495.1
12	211.8	363.3	575.1

Table 1: Total completion times of initial maps in seconds

The sum of completion times of our approach on the initial set of maps can be seen in Table 1. Group 6 performed the best, followed by our algorithm. Group 4 performs slightly worse by a little over a second. Group 17 and 12 followed with slower times. Our approach showed to have good performance both on the car and drone. In comparison to the other listed groups, our car completes the terrains slower but the drone is among the fastest.
Table 2 summarizes the performance of our approach with the car across the different competition terrains. Group 17 and 6 lead the leaderboard with fast completion times across all terrains. Our car also finishes all terrains

Group	K	L	M	N	O
17	34.9	59.6	33.9	18.8	39.7
6	42.6	70.7	42.1	17.2	45.1
24	**51.3**	**65.0**	**37.0**	**49.3**	**50.5**
4	58.9	83.1	47.8	28.4	46.0
12	57.6	DNF	38.0	26.8	DNF

Table 2: Car completion times of competition maps in seconds ordered by overall performance

given more time. It needs more time to cross terrain N because it crashes into a corner of an obstacle. The fallback method allows the car to reverse repeatedly in order to complete the map. Group 12, however, encountered difficulties on terrains L and O and did not finish. These results indicate that our modified Hybrid A* algorithm paired with our waypoint tracking system yields robust and efficient The results for the car platform validate our design choices for the parameters, the use of motion primitives and the adaptive tracking strategy.

Group	K	L	M	N	O
6	45.8	78.5	41.8	21.8	48.3
4	54.4	87.0	45.1	28.6	57.0
17	67.6	111.5	65.4	36.3	88.9
12	79.3	368.0	56.0	75.7	DNF
24	**DNF**	**DNF**	**43.6**	**24.2**	**DNF**

Table 3: Drone completion times of competition maps in seconds ordered by overall performance

Moving onto the competition results for the drone in Table 3, the order of groups is different. The two best groups 6 and 4 show fast completion times with single-digit time differences. Group 17 showed longer completion times but finished every single terrain. Group 12 showed similar behavior except for an unusually high completion time of 368 seconds in terrain L. It does not finish on terrain O. Within the selected groups for visualization, our group performed the worst by failing to reach the goal of terrains K, L and O. Our drone hit a corner of terrain K and did not find a viable path on terrains L and O. For the finished terrains, our approach yields good results, competing with the two best groups 6 and 4.

4.3 Other Group Solutions

Group 6 implemented a unified approach for both the car and drone. They used Dijkstra's algorithm as a heuristic to avoid obstacles and Hybrid A* too to generate a traversable path. They implemented post-processing methods to segment forward and backward movements while computing speed limits that take curvature into account. By integrating these acceleration constraints, they ensure a fast movement speed while staying on the path. Especially the latter post-processing method could enhance our proposed method by speed profiles across the

One particular concept from Group 17 seems to work well, namely their obstacle avoidance heuristic based on raycasts. They utilize raycasts in front of the car to check for obstacles and actively steer in towards the opposing sides. By including multiple raycasts, they are able to steer more precisely.

Group 4 adopted a similar approach to maximize the speed of the drone by trying to increase the number of long, straight lines. First, they generated a path using Informed RRT*, followed by a post-processing step that evaluates line-of-sight traversability between waypoints and removes unnecessary ones. Because of our turning penalty, we achieve a similar desired behavior of having long, straight lines.

In summary, while our experimental results confirm the effectiveness of our system for car navigation, they also underscore challenges that remain in adapting similar methods for the drone, especially in environments where one has to avoid collisions. Implementing a heuristic that avoids obstacles while traversing the environment similar to the other groups would be beneficial.

5 Analysis of Failure

Our method for the drone failed to finish the terrains K, L and O of the competition. In the following, we will discuss the technicalities of our failure while identifying the core issue and providing solutions for it.

5.1 Technical Aspect of Failure

The reason for the failure can be traced back to two main areas, mainly concerning the drone. One problem was collisions when traversing corners, as seen in Figure 2, and the other being not finding a suitable path with our approach. We identified the causes during the competition and propose solutions to address them. We start by closely examining the collisions.

Group 1 1. All the waiting is done initially 2. First car doesn't consider any other cars 3. Opp direction heuristics - usually leads to jaggered paths -

there was a lot of parameter tuning there - add each point in the dictionary and

Group 4 is more relatable - They did group 1 but with more mistakes

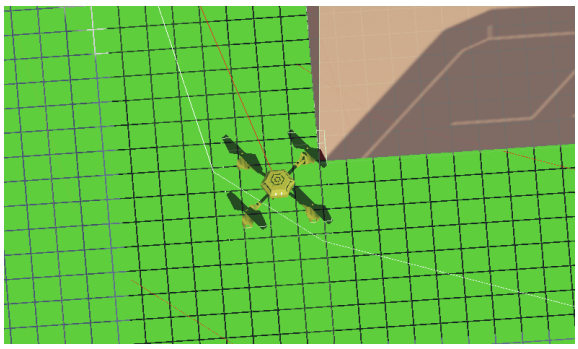


Figure 2: Drone corner crash example

5.1.1 Corner Collisions

Given our parameter selection, the drone seemed to find a feasible path but failed to follow in specific scenarios without corner collision for some maps. The underlying issue occurs when calculating the feasible path. Our implementation checks for traversability by projecting parallel lines along the vehicle's orientation and checking for obstacles along these lines. This logic is suitable for the car, as its movement orientation directly affects its rectangular collision box.

However, the drone has a fixed collision box that does not rotate to its movement direction. As a result, vertical movements require a wider path width than moving along a single axis. A simple solution is to simply increase path width, but the path planner may still fail to find suitable paths for terrains with a higher obstacle density.

We need to adjust our traversability check to account for the drone collision box. By knowing the length l and width w of the non-rotating collision box, we can compute the actual object width based on the movement angle α:

$$w_{\text{adj}} = w + (l - w) \left|\sin(\alpha)\right| \tag{2}$$

This formula returns the actual width of the object when traversing at an angle. By scaling the desired path width with the ratio w_{adjusted}/w we ensure that the fixed collision box is always accounted for, regardless of the movement direction.

5.1.2 No Suitable Paths

This leads us to the second reason for failure. Even with smaller path widths, the algorithm sometimes did not find a suitable path. The primary cause for this is the mentioned orientation binning for the node hashing in section 3. Adding more orientation bins allows the algorithm to find paths where it failed to previously. This is crucial for narrow areas with small margins for deviations in steering. Increasing the bins also leads to improved paths for previously solved terrains as well. The trade-off is a noticeable increase in computing time, which can quickly add up to hours.

To conclude, both of these issues did not appear in the practice terrains but became problematic in the competition terrains. While we identified the root causes, limited time prevented us from fully addressing them before the competition.

5.2 Management Aspect of Failure

We believe the managerial aspect of the project went as planned for the most part. We spent the aimed time and efforts on the project as depicted in Figure 3. Due to our limited experience with Unity initially, we required more time than expected to familiarize ourselves.

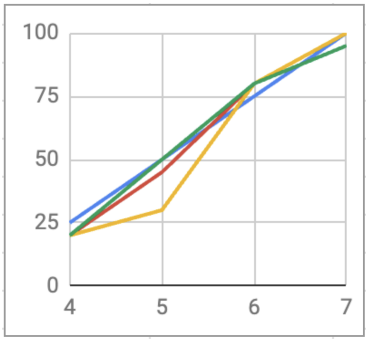


Figure 3: Progress Chart

We finished implementing an initial draft of the current algorithm by the end of week 5. We spent a lot of time with trial and errors, while trying to get Kinematic RRT* to work too. We scratched the sampling-based approach eventually. Looking back, we should have focused on one approach first and get it to work more robust.

After implementing our algorithm midway through week 6, our focus shifted towards optimizing the completion times and pathfinding system of

the existing maps, rather than testing for cases where the vehicle has to traverse narrower and more congested spaces.

The risk of failure became apparent only after the competition terrains were released. We entered the competition a sound approach in our belief, not expecting to encounter failed path planning attempts. At that stage, significantly altering our approach was neither feasible nor fair. Instead, we worked overnight trying to fine-tune the parameters of our Hybrid A* and the waypoint tracking system in the hope of solving these problems, but to no avail. To meet the deadline, our focus shifted from optimizing speed to ensuring the vehicle successfully completed most of the terrains.

5.3 Lessons Learned

We realize that our failure stemmed from prioritizing the wrong challenges and not checking for robustness in edge cases, where high precision is required. In the future, we need to design and implement our own test cases to account for such possibilities. Instead of focusing on the given terrains, we could have modified them to simulate more challenging conditions. This approach would better reflect real-world scenarios. We need to make more use of the Unity environment.

Furthermore, by analyzing successful presentations, we observed that a hybrid approach like the one from group 6 work well. Incorporating heuristics to actively avoid hitting obstacles could also have resulted in collision-free movements.

To sum it up, we need to put more emphasis in creating sound algorithms rather than optimising parameters early on. By implementing fallback heuristics, one can also account for potential errors.

6 Summary and Conclusions

The goal of this project was to create a path planning algorithm for a car and a drone that would simulate realistic scenarios with Unity. We implemented an adaptation of Hybrid A*, a planning algorithm that considers vehicle movement constraints, along with a waypoint tracking system to ensure smooth navigation. The approach was designed to create realistic and efficient paths while minimizing collisions and maximizing speed.

Tests conducted on a variety of terrains demonstrated that our approach produced reliable results for the car. However, the drone struggled in areas with lots of obstacles and narrow spaces, which occasionally resulted in failures. The main problems were collisions at sharp corners and the inability

to find viable paths due to limited orientation adjustments. Improvements, especially for the drone, can be made with a more robust traversability check implementation more bins for the heading discretization of nodes.

To conclude, these issues highlight how difficult autonomous navigation can be, especially when using similar algorithms on different types of vehicles. Because the car and drone have different kinematic restrictions, there are special planning challenges that call for more robust solutions. Our results highlight the value of environment-aware, adaptive heuristics for improving robustness. Building dependable autonomous systems requires addressing these kinds of constraints. To improve performance even more, future research can concentrate on including dynamic path re-planning and real-time obstacle avoidance.

References

[1] K. Daniel, A. Nash, S. Koenig, and A. Felner. Theta*: Any-angle path planning on grids. *Journal of Artificial Intelligence Research*, 39:533–579, October 2010.

[2] Sergey Dolgov, Sebastian Thrun, Michael Montemerlo, and James Diebel. Practical search techniques for trajectory planning of autonomous vehicles. In *Proceedings of the 2008 IEEE International Conference on Intelligent Transportation Systems*, pages 231–236. IEEE, 2008.

[3] Jonathan D. Gammell, Siddhartha S. Srinivasa, and Timothy D. Barfoot. Informed rrt*: Optimal incremental path planning focused through an admissible ellipsoidal heuristic. *CoRR*, abs/1404.2334, 2014.

[4] Daniel Harabor and Alban Grastien. Improving jump point search. *Proceedings of the International Conference on Automated Planning and Scheduling*, 24:128–135, May 2014.

[5] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. *CoRR*, abs/1105.1186, 2011.

[6] Steven M LaValle. Rapidly-exploring random trees: A new tool for path planning. In *Technical Report TR 98-11*. Iowa State University, 1998.

[7] Janko Petereit, Thomas Emter, Christian W. Frey, Thomas Kopfstedt, and Andreas Beutel. Application of hybrid a* to an autonomous mobile robot for path planning in unstructured outdoor environments. In *ROBOTIK 2012; 7th German Conference on Robotics*, pages 1–6, 2012.

[8] Rajesh Rajamani. *Vehicle Dynamics and Control*. Springer US, 2012.

[9] Drew Webb and Jur van den Berg. Kinodynamic rrt*: Asymptotically optimal motion planning for robots with linear dynamics. In *Robotics and Automation (ICRA), 2013 IEEE International Conference on*, pages 5054–5061. IEEE, 2013.